

MEDIATEK

MT8676 Hypervisor Touch User Manual

Version: 1.1
Release date: 2024-11-22

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-11-04	Pavel.Xu	Official release
1.1	2024-11-22	Pavel.Xu	Modified the Hypervisor description in Section 1.1 Overview

Table of Contents

Version History 2

Table of Contents 3

1 Touch 4

1.1 Overview 4

1.2 Touch Driver Porting 4

1.2.1 Porting Source Code 4

1.2.2 Touch Porting 5

1.3 Touch Pass-Through Configuration 6

1.4 Setting of Touch Virtualization 6

1.4.1 Touch Virtualization Flow 6

1.4.2 Setting of Touch Virtualization Code 7

1.5 Touch Common Issues 8

1.6 Relationship between Touch and Panel 9

Exhibit 1 Terms and Conditions 10

1 Touch

1.1 Overview

Typically, porting a Touch driver from a general source involves debugging according to the specifications provided by the manufacturer, with a primary focus on configuring I2C and GPIO. For panels connected via SerDes, attention should be paid to SerDes pass-through configuration. If Hypervisor systems are involved, virtualization must be used to complete the process. MT8676 has two ways to porting: the Touch Virtualization Scheme and the Touch Pass-Through Scheme. Detailed introductions are provided below.

1.2 Touch Driver Porting

1.2.1 Porting Source Code

After obtaining the Touch Vendor source code from MediaTek or a third-party vendor, the following steps are generally required to ensure that the Touch driver itself can work correctly:

1. When obtaining the source code, there may be a kernel version upgrade. In this case, you need to first port it to the corresponding version to ensure it can build successfully. The main files to be modified include the Touch driver file, deconfig, DTS, and ko_table. If specific platforms may require configuring additional files, please consult a system engineer for assistance.
2. According to the schematic, confirm the GPIO used for the interrupt/reset pin of the corresponding Touch and configure it in the DTS file.
3. Based on the compatible string in the Touch driver, write it into the DTS Touch node.

Reference DTS configuration is as follows:

```
&i2c0 {
    clock-frequency = <100000>;
    pinctrl-names = "default";
    pinctrl-0 = <&i2c0_pins>;
    status = "okay";

    ilitek@41 {
        compatible = "ilitek,touch";
        reg = <0x41>;
        interrupt-parent = <&pio>;
        interrupts = <9 0x0>;
        ilitek,irq-gpio = <&pio 9 0x0>;
        ilitek,reset-gpio = <&pio 60 0x0>;
        ilitek,vbus = "vcc_i2c";
        ilitek,name = "ilitek_i2c";
    };
};
```

As shown above,

- i2c 0: Touch mounting node location
- clock-frequency = <100000>; The clk of I2C clk, it is usually set from 100k to 400k

- compatible = "ilitek,touch"; The compatible string of Touch
- interrupts = <9 0x0>; Touch interrupt GPIO
- ilitek,reset-gpio = <&pio 60 0x0>; Touch reset GPIO

1.2.2 Touch Porting

Common debug commands:

1. With the commands as below, the trajectory line of Touch will be displayed:

```
settings put system pointer_location 1
settings put system show_touches 1
```

2. Check the log:

```
echo 1 > /proc/mtprintk
```

3. Check the status of GPIO pin:

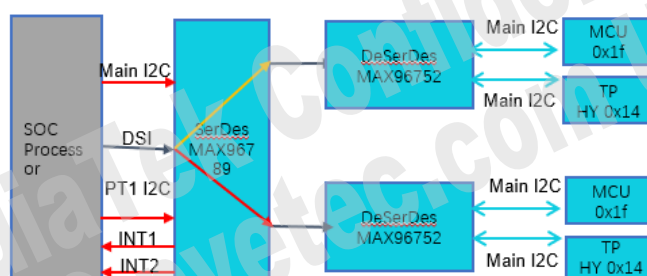
```
find -name soc.pinctrl
cd /proc/mtk_gpio
cat soc.pinctrl
```

4. Check the ko status:

```
Lsmod |grep touch_name
```

After porting the Touch source code to the platform and ensuring it can build successfully, if the Touch does not work after the platform starts, the following points generally need attention:

1. Check whether the Touch driver probe has completed running. Generally, the following types of errors may prevent it from running successfully:
 - DTS mismatch: This issue requires checking whether the Touch node is enabled in the DTS and whether the compatible string matches.
 - I2C returns -6: This error indicates that the Touch device is not recognized. You need to check whether the hardware connections are correct and whether the power supply is normal. If everything is correct, you need to measure the I2C waveform for further clarification.
 - I2C returns -110: You need to check whether the I2C part is working properly.
2. Intermittent Touch not working: Generally, the power supply voltage of the Touch has specific limits. You can print the voltage value during the Touch driver probe to clarify the issue.
3. Touch position deviation: You need to adapt the Touch according to the display size.
4. Touch module leakage issue: Determine whether the Touch actively powers off and pulls down the corresponding GPIO during suspend.
5. Check the power-on sequence of the interrupt and reset pins: Specific Touch drivers have specific requirements for the timing of these two pins, which need to be configured according to the specification.
6. Configure SerDes: Refer to the SerDes manual. The public version of MediaTek now supports 6-panel touch in the following figure.



The reference SerDes configuration is as follows, and you need to refer to 96789 and 96752 manuals for the corresponding configuration.

```
superframe_setting_hy: setting3 {
    ser-super-frame = <1>;
    ser-init-cmd = <
        0x02dc 0x04 0x00 /* 96789 INT: des gpio4-> ser gpio10*/
        0x02dd 0xaa 0x00 /* 96789 huayang tp ser int GPIO10 */
        0x02de 0x6a 0x00 /* 96789 huayang tp ser int GPIO10 */
        0x02d9 0x04 0x00 /* 96789 INT: des gpio4 -> ser gpio9 */
        0x02da 0xa9 0x00 /* 96789 huayang tp ser int GPIO9 */
        0x02db 0x69 0x00 /* 96789 huayang tp ser int GPIO9 */
    >;
```

1.3 Touch Pass-Through Configuration

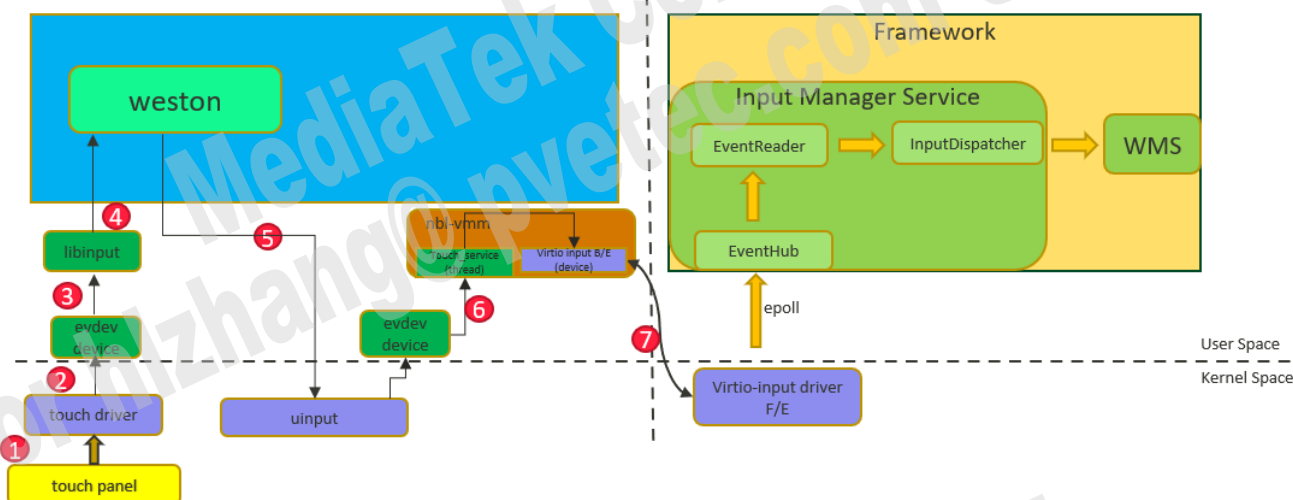
Touch pass-through configuration means configuring the Touch driver on the Yocto and Android sides as needed. This depends on the completion of GPIO, I2C and EINT virtualization. Once the underlying virtualization is completed, the Touch driver can be configured without considering the virtualization part, and you can simply repeat the steps for configuring Touch in a single system.

1.4 Setting of Touch Virtualization

1.4.1 Touch Virtualization Flow

For Touch security, the Touch driver is placed on the Yocto side. In this case, Touch virtualization is required. The general flow diagram of the public version of Touch virtualization is as follows:

Yocto Host Android Guest



1. Yocto acts as the host side, with the Touch driver placed on the Yocto side. The Touch input device node "mtk-tpd-yocto" is registered and created on the Yocto side.
2. The Hypervisor SDK starts services to poll and monitor "mtk-tpd-yocto" for events, retrieves events from "mtk-tpd-yocto", and transmits the events to the Android side via virtio-input.
3. The Android side reports the transmitted Touch events to the upper framework for processing.

1.4.2 Setting of Touch Virtualization Code

With the touch driver already configured on the Yocto side, the following configurations need to be added to set up Touch virtualization:

1. Enable Touch Virtualization
 - The path for change:
*`\src\hypervisor\grt_mt8676\thyp-sdk\vm\mb1_vmm\BUILD.bazel`
 - The location of the code:

```
cc_binary(  
    name = "nbl_vmm_mix",  
    srcs = [  
        "main.cc",  
    ],  
    deps = [  
        ":input",  
    ]  
)
```

2. Based on the Touch node information on the Yocto side, configure the Touch nodes that need to be forwarded.
 - The path for change:
*`\src\hypervisor\grt_mt8676\thyp-sdk\products\mt8676-mix\guest-configs\uos_alps_pv8676.lua`

- The location of the code:

```
-- input
uos_config:setInput ("/dev/input/event2")
```

3. Configure the raw touch size for virtualized Touch.

- The path for change:

```
*\src\hypervisor\grt_mt8676\thyp-sdk\products\mt8676-mix\guest-
configs\uos_alps_pv8676.lua
```

- The location of the code:

```
-- display
--[[
void setDisplayParam(int id, int x, int y,int has_touch, int type)
type: VIRTIO_INPUT_MOUSE:1; VIRTIO_INPUT_SINGLE_TOUCH:2; VIRTIO_INPUT_MULTI_TOUCH:3
--]]
uos_config:setDisplayParam(0,720,800,1,2)
```

4. Please note the following:

- Currently, touch virtualization can support up to 4 virtual nodes.
- Currently, touch virtualization transmission only supports Type A protocol.

1.5 Touch Common Issues

1. Touch virtualization is not effective.

Enter `getevent -l` on both Yocto and Android sides and touch the screen. For example:

- If no Touch data is generated on the Yocto side, check the source Touch driver configuration.,
- If no Touch data is generated on the Android side, check whether Touch virtualization is enabled and whether the corresponding touch node name is set.

2. There is a deviation between the Touch point position and the display.

Enter the following command on the Android side to display the Touch trajectory lines and check for any deviation and its pattern.

```
settings put system pointer_location 1
settings put system show_touches 1
```

Enter `getevent -I` on the Android side to check if the Touch `raw_size` corresponds to the display size. If not, adjust the `display_size` information in NBL.

3. When multiple Touch devices need to be virtualized, incorrect data may be used in the Touch `raw_size`.

Ensure that when configuring `uos_alps_pv8676.lua`, the order of `display_size` matches the order in which the Touch drivers are loaded.

1.6 Relationship between Touch and Panel

As shown in the figure below, the input module binds the Touch node to the panel using the location information of the node. When other nodes are generated before the Touch node, the location number may change, causing the binding to fail. To ensure the following through the ko_table, any subsequent ko that generates nodes should be placed after the Touch node to avoid binding issues.

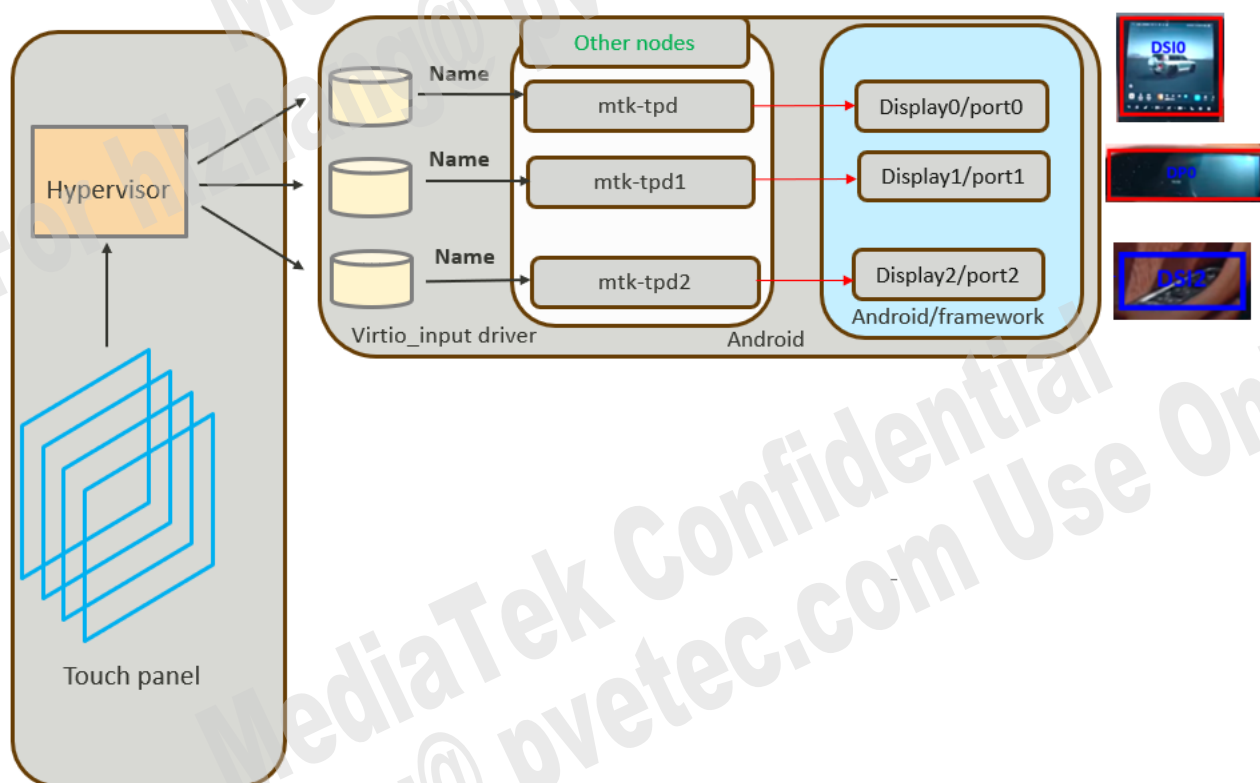


Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.